تمارين على العوامل المنطقية، الإزاحة والتعليقات

الدرس 10 — 1IOT، SST-ICT-01 — السنة الكبيسة، المثلث الصالح، الإنذار بـNOT، الإزاحة، التعليقات؛ مقدمة في firmware (Arduino، Harvard وVon Neumann)

12/10/2026

1 من أين ننطلق من جديد

ملاحظة

بعد **الاختبار** **1** صباحًا، يطبق هذا الدرس **العوامل** **المنطقية** من الدرس السابق: **AND** (&&) و**OR** (||) و**NOT** (!) داخل **شرط** `if`. تُكتب برامج كاملة في **onlinegdb**، انطلاقًا من نصوص تمارين الورقة **"V02 - Esercizi Operatori Logici"**. ثم تُقدَّم **قاعدتان** **للكتابة** (الإزاحة والتعليقات)، وفي الدقائق الأخيرة أول **حديث** عن firmware.

تعريف

قاعدة الترجمة من الجمل إلى العوامل. عند قراءة نص تمرين: إذا وجدت **"oppure"** (أو) فهذا يعني **OR**، أي الخطان العموديان ||؛ وإذا وجدت **"e"** (و) فهذا يعني **AND**، أي &&؛ وإذا وجدت **"non"** (لا) فهذا يعني **NOT**، أي علامة التعجب !. بالإنجليزية "e" تعني **and**. و**"diverso"** (مختلف) يُكتب بعلامة التعجب متبوعة بعلامة المساواة، =!، وليس بـ= وحدها.

2 تمرين: السنة الكبيسة

مثال

النص (التمرين 10 من الورقة). اكتب برنامجًا بلغة C يطلب سنة. إذا كانت السنة تقبل القسمة على 400 **أو** (تقبل القسمة على 4 **ولا** تقبل القسمة على 100)، يطبع البرنامج "Anno bisestile" (سنة كبيسة)، وإلا يطبع "Anno non bisestile" (سنة غير كبيسة). **ملاحظة: استخدم العامل % لقابلية القسمة، مثل `anno % 4 == 0`.**

الخطوة 1: متغير السنة

ما **نوع** **البيانات** اللازم لاحتواء سنة مثل 2026؟ يفكر الأستاذ بصوت عالٍ مع القسم، مستبعدًا البدائل:

- `char` (في هذه الدورة 0–255): **لا**، 2026 لا يتسع فيه؛
- `float`: **لا**، السنة بلا فاصلة عشرية، ولا معنى لذلك؛
- `int` هو **الخيار** **المعقول** **الوحيد**.

إذن `int anno;`. ثم تُطلب القيمة من المستخدم بـ`printf` ("Inserisci l'anno") وتؤخذ بـ`scanf` باستخدام `%d` و`&anno`.

الخطوة 2: الشرط

يجب أن يختار البرنامج بين طباعة "bisestile" و"non bisestile": نحتاج إلى **if-else**. يُبنى الشرط قطعة قطعة.

1. **يقبل القسمة على 400**: يقبل العدد القسمة على 400 إذا كان **باقي القسمة** (**العامل** %) بين anno و400 **صفرًا**، أي (anno % 400 == 0). وتوضع الأقواس "للأمان".
2. يقول النص "**أو**": نحتاج إلى العامل **OR**، ||، وبعده شرط آخر بين قوسين.
3. **يقبل القسمة على 4**: (anno % 4 == 0).
4. يقول النص "**و**": نحتاج إلى العامل **AND**، &&.
5. **لا يقبل القسمة على 100**: إذا لم يقبل العدد القسمة فإن الباقي **مختلف عن الصفر**، أي (anno % 100 != 0).

مثال

الكود المكتوب في القسم (مع التعليق الأول الذي فيه نص التمرين، كما نُسخ من **onlinegdb**، لذلك هو بالإيطالية):

```c
/*
Anno Bisestile: Scrivere un programma in C che richieda un anno.
Se l'anno e' divisibile per 400 OPPURE (divisibile per 4 E NON divisibile per 100),
il programma stampera "Anno bisestile", altrimenti stampera "Anno non bisestile".
(Nota: usa l'operatore % per la divisibilita', es. anno % 4 == 0)
*/
#include <stdio.h>

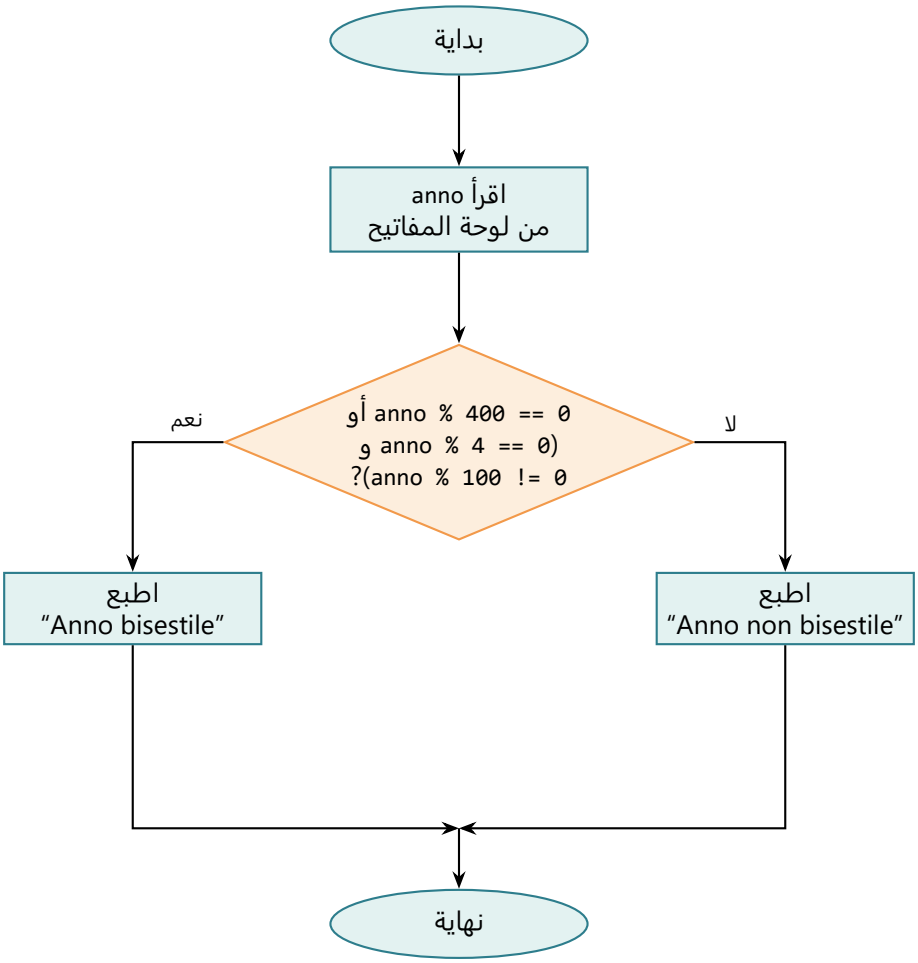
int main()
{
    int anno;
    printf("Inserisci l'anno: ");
    scanf("%d", &anno);

    if ( (anno % 400 == 0)  ||  ( (anno % 4 == 0) && (anno % 100 != 0) ) )
    {
        printf("Anno bisestile");
    }
    else
    {
        printf("Anno non bisestile");
    }

    return 0;
}
```

ملاحظة

تجربة البرنامج. جرب القسم البرنامج بعدة سنوات: **2000** كبيسة (تقبل القسمة على 400)؛ و**2025** و**2026** **ليستا** كبيستين؛ وكذلك 2022. للسنة الكبيسة **يوم إضافي** (فبراير 29 يومًا). خطأ كتابة من الأستاذ (101 % بدلًا من 100 %) نبّه إليه القسم وصُحح: **تحقق دائمًا من الأعداد** المكتوبة في الشروط.



شكل 1: مخطط انسيابي للسنة الكبيسة: قرار واحد، بشرط مركب من OR وAND.

تنبيه

لا تُترك قاعدة "**AND** أولًا، و**OR** بعدها" للصدفة: **الأقواس** حول كل **شرط** **بسيط** وحول كتلة AND تجعل القراءة فورية. انظر قسم أولوية العوامل لاحقًا.

3 قاعدة جديدة: الإزاحة

تعريف

الإزاحة (**indentazione**) هي **دفع** الكود إلى اليمين بشكل منظم. كتبها الأستاذ على السبورة ("نسيت حرف **INDENTAZIONE**": N) ومن اليوم هي **إلزامية**. القواعد:

- **القوس المعقوف** الذي يغلق يقع **عموديًا تمامًا** تحت القوس الذي يفتح، في العمود نفسه؛
- كل الكود **الموجود** بين القوسين يُكتب بمقدار **جدولة** واحدة إلى اليمين، في كل سطر: اضغط المفتاح **TAB** (يسار Q)؛
- إذا وجدت أقواس أخرى في الداخل، يُدفع محتواها جدولة أخرى إلى اليمين.

يدخل المفتاح TAB بعض الفراغات (عادة ثلاثة، وفي المحرر المستخدم في القسم اثنان أو أربعة): ولا يمكن ضبطه في محرر **onlinegdb**.

مثال

الكود نفسه **بدون** إزاحة و**معها**. في النسخة الأولى لا يُرى أن قوسًا ناقص؛ وفي الثانية يُرى فورًا. (الكود من اليسار إلى اليمين دائمًا.)

```
if (a < b)
{
printf("minore");
}
else
{
printf("maggiore");
```

```
if (a < b)
{
    printf("minore");
}
else
{
    printf("maggiore");
```

مع الكود المزاح يُفهم فورًا أن **القوس ناقص** في النهاية لإغلاق `else`: "أفتح، أفتح، أغلق، ويجب أن أغلق ذاك أيضًا". أما في الكود الملتصق باليسار فالخطأ غير مرئي.

تنبيه

لماذا هي إلزامية. يعمل الكود **على** **أي** **حال** بدون إزاحة (في C "نحن محظوظون")، لكنه غير مقروء. في الشركات يُعمل ضمن فريق: يجب أن يتمكن **الزملاء** من قراءة الكود وفهم ما تم. ملف من 15 000 سطر بلا إزاحة غير مقروء ويجعل **الصيانة** بطيئة ومليئة بالمشاكل. من اليوم أي كود مسلَّم **بلا** **إزاحة** **قيمته** **صفر**: "لا أرغب في ذلك فقط، بل أشترطه".

- خطأ معتاد: سطرا `printf` مدفوعان **كثيرًا** **إلى** **اليمين** مقارنة بحاوية الأقواس. يعمل، لكنه خطأ في الأسلوب.
- في لغات أخرى مثل **Python** لا توجد أقواس معقوفة: تتحدد مجموعة التعليمات **بالإزاحة فقط**، والإزاحة الخاطئة تجعل البرنامج **لا** **يعمل**.

4 تمرين: المثلث الصالح

مثال

النص (التمرين 9 من الورقة). اكتب برنامجًا بلغة C يطلب ثلاثة أعداد صحيحة لأضلاع مثلث (a وb وc). إذا كان `a < b + c` **و** `b < a + c` **و** `c < a + b`، يطبع البرنامج "Triangolo valido" (مثلث صالح)، وإلا يطبع "Triangolo non valido" (مثلث غير صالح).

تعريف

الطريقة: **أعاد** الأستاذ **كتابة** **النص** محولًا إياه إلى كود: "إذا" تصبح **`if`**، و"و" تصبح **AND** (`&&`). تُعرَّف ثلاثة **متغيرات** **صحيحة**، وتُطلب الأضلاع الثلاثة بـ`printf` و`scanf` (النصوص ضرورية، "وإلا فلا يُفهم شيء")، ثم يُكتب الشرط. التعريفات و`printf` و`scanf` تخص أولًا a، ثم b، ثم c.

مثال

الكود المكتوب في القسم:

```c
#include <stdio.h>

int main()
{
    int a;
    int b;
    int c;

    printf("Inserisci il primo lato del triangolo: ");
    scanf("%d", &a);
    printf("Inserisci il secondo lato del triangolo: ");
    scanf("%d", &b);
    printf("Inserisci il terzo lato del triangolo: ");
    scanf("%d", &c);

    if ( ( a < b + c)  && (b < a + c)  && (c < a + b) )
    {
        printf("Triangolo valido");
    }
    else
    {
        printf("Triangolo non valido");
    }

    return 0;
}
```

ملاحظة

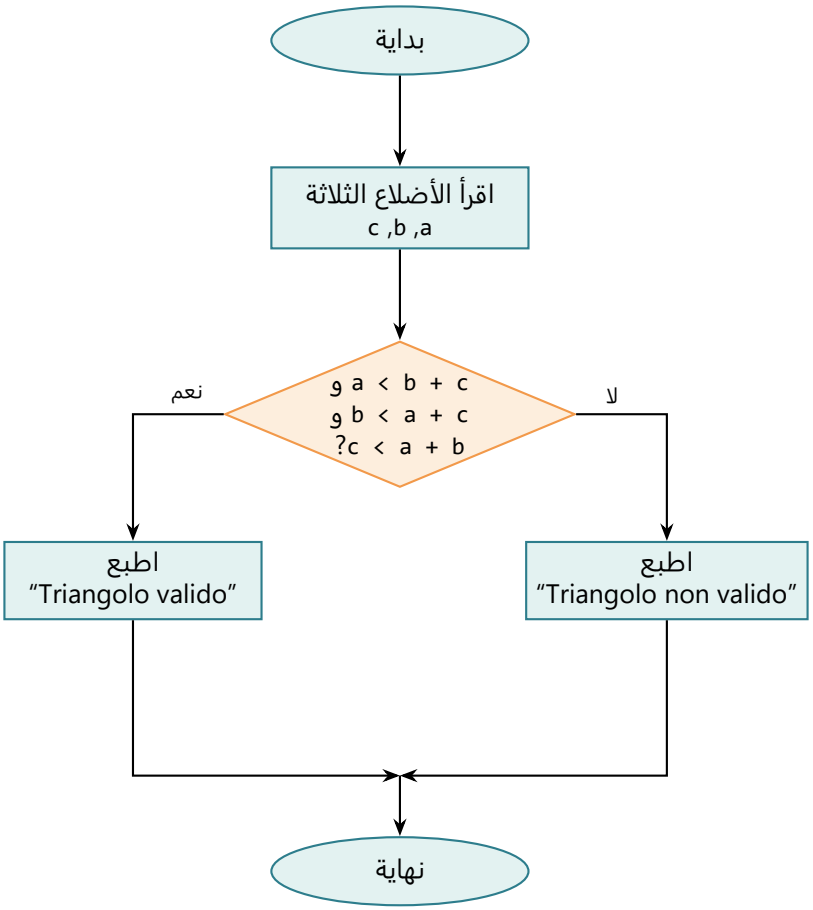
لماذا هذا الشرط. في المثلث يجب أن يكون كل ضلع **أصغر من مجموع** الضلعين الآخرين. يتحقق البرنامج من المتراجحات الثلاث معًا: يكفي أن تكون **واحدة فقط** خاطئة ليعطي **AND خطأ** ولا يكون المثلث صالحًا. (إذا كان ضلع **يساوي** مجموع الآخرين "ينسحق" المثلث إلى قطعة مستقيمة ويستبعده الشرط الصارم، بشكل صحيح. ولا يطلب النص التحقق من أن الأضلاع موجبة.)

الأقواس وأولوية العوامل

يمكن كتابة الشرط أيضًا **بدون** الأقواس الداخلية (a < b + c && b < a + c && c < a + b) وقد عمل. يقول الأستاذ: يعمل، لكنه **خطر**. يقيّم المترجم أولًا **الجمع**، ثم **المقارنات**، وأخيرًا العامل المنطقي **AND**. قد يظن البعض أن الترتيب يتغير بتغيير المترجم: "لقد كنت محظوظًا". يضع الأستاذ الأقواس **دائمًا**، لأنه هكذا "لا مجال للخطأ"، وإذا أراد مزيدًا من الدقة يضعها حول الجمع أيضًا، (b + c). وهكذا يعمل الكود مع كل المترجمات. "نحن نعمل على Arduino وبلغة C، ولا نتجاوز ذلك."

تنبيه

للدقة: **ترتيب التقييم** (**الأولوية**) للعوامل يحدده **معيار لغة C**، ولا يتغير من مترجم إلى آخر: أولًا العوامل الحسابية، ثم المقارنات، ثم **&&**، ثم **||**. وتبقى قاعدة الأستاذ العملية صحيحة: **الأقواس** تزيل كل شك عند من يقرأ الكود (ومن يكتبه).



شكل 2: مخطط انسيابي للمثلث الصالح: ثلاث مقارنات يجمعها AND مرتين.

5 تمرين: الإنذار بالعامل NOT

مثال

النص (ورقة التمارين، منسوخ في المحرر). Allarme Disattivato: اكتب برنامجًا بلغة C يطلب من المستخدم حالة الإنذار (1 مفعل، 0 معطل). باستخدام العامل NOT (!)، إذا كان الإنذار غير مفعل يطبع البرنامج "Casa sicura" (البيت آمن)؛ وإلا "Allarme in corso" (الإنذار يعمل) (النص في المحرر مقروء جزئيًا فقط).

المتغير والإدخال

لحفظ 1 أو 0 يكفي **char**: لا حاجة إلى `int`. يكتب الأستاذ `char allarme;`، ثم `printf` ("Inserisci lo stato (1 attivo, 0 disattivato)") و`scanf` مع `%d` و`&allarme`. يمكن استخدام `int` أيضًا: "لا بأس، لا يتغير شيء"، لأن ما يهم فورًا هو تقييم هل الإنذار مفعل أم معطل، **صحيح أم خطأ**. (حالة أكثر تقدمًا هي keypad، لوحة الأزرار الصغيرة: تتطلب قراءة الأزرار، وهو موضوع يأمل الأستاذ الوصول إليه قرب نهاية السنة.)

تنبيه

ملاحظة تقنية: `%d` يقرأ عددًا **صحيحًا** (عادة 4 بايت) ويكتبه في المتغير؛ فإذا كان المتغير **char** (1 بايت) تذهب الكتابة **إلى ما بعد** المتغير، والسلوك غير محدد في المعيار. التنسيق الصحيح لـ`char` هو `%hhd`. لذلك فإن المتغير في الكود أدناه **int**، وهو خيار قبله الأستاذ ("لا يتغير شيء").

الشرط باستخدام NOT

إذا كان الإنذار **غير** مفعل ففي المتغير **صفر**. بالعامل NOT: إذا كانت قيمة `allarme` هي 0 فإن `!allarme` تساوي **1**، أي صحيح. يكتب الأستاذ الشرط `!allarme != 0`، أي "NOT الإنذار **مختلف** **عن** **الصفر**": وهذا يحل المسألة. كان يمكن كتابة `1 ==` أيضًا، لكن "عندما أستخدم NOT فإنه يجعله صحيحًا ببساطة: لا يوجد مكان مكتوب فيه أنه يجعله واحدًا، أو على الأقل ليس دائمًا". وكان يمكن استخدام **أكبر** **من** **الصفر**، لكن **مختلف** **عن** **الصفر** أفضل: "إنه أسهل، لأن العدد يمكن أن يكون موجبًا أو سالبًا، كما تشاء".

مثال

البرنامج **الكامل** (أُعيد بناؤه من الشرح؛ النص المكتوب في القسم غير ظاهر كاملًا في اللقطات):

```c
#include <stdio.h>

int main()
{
    int allarme;

    printf("Inserisci lo stato dell'allarme (1 attivo, 0 disattivato): ");
    scanf("%d", &allarme);

    if ( (!allarme) != 0 )
    {
        printf("Casa sicura");
    }
    else
    {
        printf("Allarme in corso");
    }

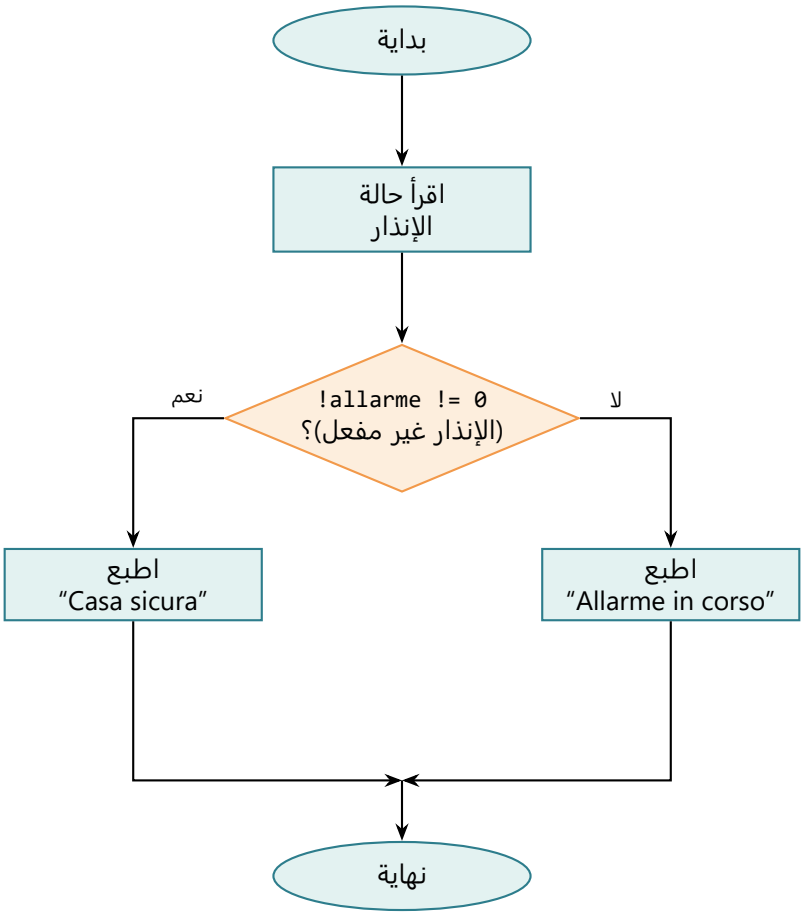
    return 0;
}
```

كتابة **firmware**: `!allarme` بلا مقارنة

يمكن كتابة الشرط أيضًا **بدون** المقارنة مع الصفر، بترك `!allarme` فقط، وهو يعمل. لماذا؟ ليس لأنه "يعكس على أي حال": إنها طريقة يستخدمها كثير من **مبرمجي** **firmware**، "وأنا أولهم أكتب هكذا"، أي **كتابة** **أقل**. مع القسم كتب الأستاذ **مختلف** **عن** **الصفر** لأن الأساسيات تأتي أولًا، لا الطرق المتقدمة.

تعريف

يجب أن يعطي **شرط** `if` جوابًا **صحيحًا** **أو** **خطأ**. تنتج العوامل الشرطية (`a > b`، `x != 0`، ...) صحيحًا أو خطأ؛ لكن `!allarme` أيضًا يعطي وحده صحيحًا أو خطأ. إذن **لا** **حاجة** إلى المقارنة مع الصفر: `if (!allarme)` تكفي. والقاعدة التي تحتها سبق أن رأيناها: **الصفر** **=** **خطأ**، **أي** **عدد** **آخر** **=** **صحيح**.



شكل 3: مخطط انسيابي للإنذار: يستخدم الشرط العامل NOT.

تنبيه

مثال على الخطر: إذا كان في المتغير 42، فإن 42 **صحيح** وبنفيه يصبح **خطأ**: ينفذ البرنامج دائمًا فرع `else`. لا نقول "موجب" أو "سالب": نقول **صحيح** أو **خطأ**. الكتابة المختصرة تعمل جيدًا جدًا، لكن "هذا شيء أكثر تطورًا قليلًا: وخطر سوء تفسيره مرتفع جدًا". ما لم تكن متأكدًا تمامًا، **انتبه**: "الخطأ قريب دائمًا".

الأقواس المعقوفة اختيارية مع تعليمة واحدة

إذا وجدت بعد `if` أو `else` **تعليمة** **واحدة** **فقط**، يمكن حذف **الأقواس** ويعمل البرنامج. ونحن **نضعها** **على** **أي** **حال**، لأن الكود يكون أكثر قابلية للقراءة: "تجنبوا هذه الأشياء، إن أمكن".

ملاحظة

تقييم التمارين. كيف ستُقيَّم تمارين البرمجة؟ الاختبار الذي يطلب "أجب عن هذه الأسئلة" له معايير قليلة؛ أما **الاختبار** **القادم** فسيكون "**اكتب** **برنامجًا** يفعل هذا وهذا وذاك". البرنامج إما يعمل أو لا يعمل، لكن في C قد يعمل أيضًا **بالصدفة** (رأينا ذلك مع الأقواس). يريد الأستاذ إعطاء مساحة للقدرة على كتابة البرنامج، لكن الإزاحة والجودة تُحسبان أيضًا. إذا كان الاختبار **على** **الحاسوب**، فتسليم برنامج **لا** **يُترجم** أمر ثقيل: يعرض المترجم الخطأ، وعدم القدرة على تصحيحه يعني عدم فهم كيفية عمله؛ أما على الورق فالفاصلة المنسية أقل وزنًا. "الدرجة واجب مؤسسي علينا."

6 التعليقات في الكود

تعريف

التعليق (**commento**) نص مكتوب في الكود **لا** **يأخذه** **المترجم** **في** **الاعتبار**. يفيد من يقرأ الكود ليفهم شيئًا منه. في C يُكتب **بطريقتين**:

- **تعليق** **متعدد** **الأسطر**: بين **/*** و**/*. كل ما بين الرمزين تعليق، حتى على عدة أسطر؛
- **تعليق** **على** **سطر** **واحد**: بعد **//**. يسري حتى نهاية السطر فقط: وفي السطر الجديد يعود **الكود** **العادي**.

مثال

على **السبورة** رسم الأستاذ الرموز: /* (فتح)، */ (إغلاق) و// وبجانبها سطر. في محرر **onlinegdb** يتغير **لون** كل النص المعلَّق (برتقالي): **التعليق** **متعدد** **الأسطر** في البرنامج (كما في ترويسة `main.c`) يبدأ من السطر 1 وينتهي في السطر 8 بـ*/.

```c
/* هذا تعليق
   على عدة أسطر: كل ما
   أكتبه هنا يتجاهله المترجم */

// أطبع رسالة
printf("Ciao");

// أطلب من المستخدم عددًا
scanf("%d", &x);
```

بيّن الأستاذ أنه يمكن وضع تعليقات قبل `printf` (”أطبع رسالة“) أو قبل `scanf` (”أطلب من المستخدم...“): **اكتبوا** **ما** **تشاؤون**، وفي نهاية السطر نعود إلى كتابة الكود. أما إذا فتحنا بـ/* فكل شيء يصبح تعليقًا **حتى** **نغلق** بـ*/.

تنبيه

التعليقات في الشركات إلزامية. من يكتب برنامجًا بلا تعليقات **يعيده** إليه صاحب العمل (”انظر، ضع التعليقات، وإلا لن أتقدم“): فغدًا قد لا يعود من كتبه يعمل هناك، والكود **ملك** **الشركة**، التي يجب أن تواصل بيعه وصيانته. ومن يأتي بعده، بلا تعليقات، لا يعرف ما هو مكتوب.

7 نغير الموضوع: ما هو firmware

ملاحظة

يفتح الأستاذ **”حديثًا** **من** **20** **دقيقة“** عن المادة الثانية في الدورة: **firmware**. حتى الآن، في مادة البرمجة، عملنا **على** **الحاسوب**. أما firmware فيعني **كتابة** **برامج** لا تعمل على الحاسوب، بل تشغل **لوحات** **إلكترونية**، مثل التي عُرضت في اليوم الثاني من المدرسة. في السنة الأولى نستخدم **Arduino**؛ ويعتزم الأستاذ تركه قبل نهاية السنة إلى شيء أكثر إثارة.

مثال

لماذا Arduino تعليمي فقط. لا يحب الأستاذ Arduino لأنه "يبسط الأمور أكثر من اللازم": هو سهل إلى درجة أنه لا يحتاج إلى كفاءة، كخلاط منزلي. إنه جيد **للتجريب** وفهم مفاهيم كانت ستكون معقدة جدًا، لكنه **ليس صنع firmware** **رسميًا** **عام** **2005** (مع نماذج أولية قبل ذلك): تُوصل بالحاسوب **بكابل USB**، ويُكتب لها ما تفعله فتفعله.

قبل ذلك كان الأستاذ يعمل مع الشرائح السوداء الصغيرة فقط، أي **المتحكم الدقيق** ("دماغ" اللوحة)، بلا لوحة: كان يجب شراء **الدليل** (نحو 70–80 ألف ليرة، أي 80 يورو اليوم)، ودراسته، وكتابة نحو 200 سطر من الكود **فقط** لضبط الآلة، ثم التركيز على ما تفعله. كانت صعوبة البداية عالية جدًا: الإنترنت قليل، وYouTube غير موجود، ولا أحد يشرح. مع هذه اللوحة صار كل شيء بسيطًا، لكن جرب أن تفتح **الثلاجة** أو **الفرن** أو **الميكروويف** أو التلفاز أو الهاتف: لن تجد خلفها لوحة كهذه. إنها جيدة على المستوى **التعليمي**، لا على مستوى **الإنتاج**. ومن يتعلم Arduino فقط طوال أربع سنوات "لن يوظفه أحد".

تعريف

المتحكم الدقيق (**microcontrollore**) **شريحة** **سوداء** لها **أرجل** معدنية كثيرة. كل رجل هي **وصلة كهربائية**، أي سلك يمكن للآلة تشغيله أو إطفاؤه: إذا أعطيت تيارًا فهو **1**، وإذا قطعته فهو **0**. وبحسب مستويات التشغيل والإطفاء على الأرجل يحدث شيء **في الخارج**. وداخل الشريحة يُحمَّل **البرنامج** الذي نكتبه و**البيانات** المراد معالجتها.

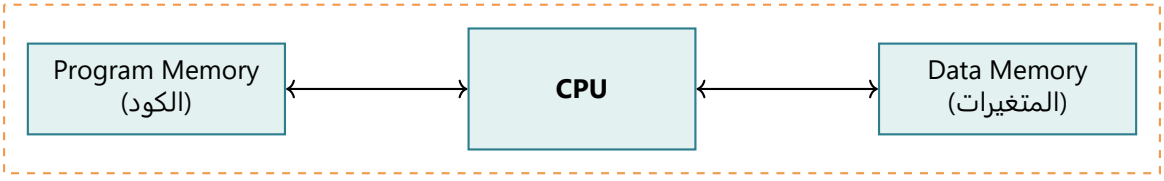
تعريف

CPU = **Central Processing Unit**، **وحدة** **المعالجة** **المركزية**: "الدماغ"، الذي يجري **الحسابات** ويختار أي تعليمة ينفذ. في الشريحة تتصل **بذاكرتين**:

- **Program Memory** (ذاكرة البرنامج): تحتوي **الكود** الذي كتبه المبرمج؛
- **Data Memory** (ذاكرة البيانات): تحتوي **البيانات**، أي كل **المتغيرات**. في كل مرة يُنشأ فيها متغير (`int x;`، `int allarme;`) ينتهي في ذاكرة البيانات.

في **onlinegdb** نكتب الكود ونظن أنه ينتهي هناك، لكن المتغير الحقيقي يُنشأ في ذاكرة بيانات الجهاز.

متحكم دقيق (بنية Harvard)



شكل 4: مخطط كتلي لمتحكم دقيق: CPU، وذاكرة للبرنامج وذاكرة للبيانات، منفصلتان.

ملاحظة

سؤال من القسم، لم يُشرح: توجد تقنية تنقل البيانات في الذاكرة دون إشغال CPU. يقول الأستاذ: "معقد جدًا الآن"، وهو أمر **متخصص** جدًا (ليس موضوع السنة الأولى)؛ وربما نتحدث عنه في الاستراحة أو في وقت آخر.

8 كم من الذاكرة؟ مقارنة مع الحاسوب الشخصي

مثال

الحاسوب المنزلي. في حواسيب الطلاب: **RAM** بسعة 32 و128 جيجابايت (128 جيجابايت = 128 متبوعة بتسعة أصفار من البايتات)؛ و**قرص** من 500 جيجابايت حتى 4 و6 و12 تيرابايت. أما في **Arduino** فذاكرة البيانات **2** **كيلوبايت** فقط. راجع الأستاذ **datasheet** (الورقة التقنية) للمتحكم الدقيق في Arduino Uno، وهو **ATmega328P** من Atmel (الآن Microchip)، من موقع الشركة المصنعة: **32** **KB** من ذاكرة **flash** للبرنامج، و**1** **KB** من **EEPROM**، و**2** **KB** من **SRAM** الداخلية للبيانات. "هل هذا كثير أم قليل؟" ذاكرة الحاسوب **أكبر** **بما** **لا** **يقاس**: "نسب عمل مختلفة تمامًا".

	(Arduino (ATmega328P	الحاسوب المنزلي (أمثلة القسم)
ذاكرة البيانات (RAM)	(KB (SRAM 2	GB 128–32
ذاكرة البرنامج / القرص	(KB (flash 32	(قرص) TB 12–4

9 بنية Harvard وبنية Von Neumann

تعريف

الجهاز الذي له **ذاكرتان** **منفصلتان**، واحدة **للبرنامج** وواحدة **للبيانات**، له **بنية** **Harvard**. إنه خيار تصميم ("يجب أن نبني نوعًا جديدًا من الحواسيب: لنجعله هكذا").

تعريف

الجهاز الذي فيه CPU و**ذاكرة** **واحدة** **فقط** تحتوي **البيانات** و**البرامج** معًا له **بنية** **Von Neumann**. مثال: **الحواسيب** **الشخصية**. البرنامج المحفوظ على القرص يُحمَّل في **RAM** ليعمل: والذاكرة التي تتبادل معها CPU البيانات هي RAM فقط.

مثال

عند العمل على **الحاسوب** **الشخصي** (onlinegdb) تكون البنية **Von Neumann**؛ وعند العمل على **Arduino** وحده تكون البنية **Harvard**. **سؤال اختبار**: "ما الفرق بين آلة من نوع Harvard وأخرى من نوع Von Neumann؟" الجواب: **Harvard** لها **ذاكرتان**، واحدة للبيانات وواحدة للبرنامج؛ و**Von Neumann** لها **ذاكرة** **واحدة** فيها كل شيء.

ملاحظة

أيهما الأفضل؟ Harvard **أسرع**: يمكن لـCPU تبادل البيانات مع ذاكرة البرنامج وذاكرة البيانات **في** **الوقت** **نفسه** (تحمّل البرنامج أثناء العمل على البيانات). أما في Von Neumann فإما يُقرأ **البرنامج** وإما تُقرأ **البيانات**: أي نعمل، بمعنى ما، "بنصف السرعة".

10 لماذا لا تكون الحواسيب الشخصية Harvard: خيارات السوق

مثال

لماذا إذن لا تُبنى الحواسيب الشخصية ببنية Harvard؟ حاسوب Harvard سيكلف **أكثر بكثير**؛ "وعندها لن تشتريه، وإذا لم تشتره فلن أنتجه". **الخيارات التقنية** ليست الأفضل أبدًا: بل هي التي **تباع**: إذا كان المنتج غالي الثمن جدًا فلا معنى لصنعه. ولو كلف حاسوب "entry level" سبعة آلاف يورو لضاعت كل **حصة السوق**.

مثال

حالة المعالجات (مثال فئات السرعة). معالج Intel i7، من الجيل نفسه وبعدد **core** نفسه، يوجد بنسخ 3 و4 و5 GHz. هل لدى Intel خط إنتاج لكل سرعة؟ **لا**: الخط واحد و**ينتج** معالجات مصممة لـ**5 GHz**. وعند خروج كل شريحة من المصنع **تُختبر**:

- إذا وصلت إلى 5 GHz، يُطبع عليها 5 GHz وتباع بسعر أعلى؛
- إذا توقفت عند 4 GHz ("معيبة ولا تستطيع الوصول إلى 5")، يُطبع عليها 4 GHz؛
- إذا توقفت عند 3 GHz، يُطبع عليها 3 GHz وتباع بسعر أقل.

هدر صفر: كل ما يُنتج يباع. من يشتري النسخة الأبطأ يشتري تقنيًا منتجًا "معيبًا" يعرف المنتج عيبه: عند 3 GHz لا يظهر ذلك، لكن عند تجربة **overclock** "تتجمد" الآلة أسرع. هذا يصح للعائلة **نفسها**: أما Ryzen 7 و9 Ryzen فهما **خطان مختلفان** (عمليتان مختلفتان قليلًا). وعن أسماء أخرى (مثل طرازات "KF") يقول الأستاذ إنه لا يعرف: فهذا عالم لا يهمه. المهم هنا أن **خط الإنتاج** واحد وأن فرق السرعة **فرز** بحسب جودة الشريحة ("جاءت جيدة؟ نبيعها بسعر أعلى").

11 الواجبات والتسليمات

تنبيه

تقرير عن Von Neumann: ينتظر الأستاذ قبل **الجمعة** أو **الاثنين القادم تقريرًا قصيرًا** عن **ما فعله Von Neumann** ("لا أدري، إذا كان يهمكم فابحثوا عنه"). كان الأستاذ يود عرض فيلم، لكنه لا يملكه. واجبات أخرى: "غدًا صباحًا لا شيء لديكم حاليًا" و"كل الكود ستجدونه في ملزمة هذا المساء": تمارين هذا الدرس (السنة الكبيسة، المثلث، الإنذار) من الورقة **V02 — Esercizi Operatori Logici**، التي تستمر ببنود أخرى (5 حرارة الإنذار، 6 لعبة في مدينة الملاهي، 7 عدد زوجي وموجب، 8 الدخول بكلمة مرور وPIN).

مثال

النصوص الدقيقة للبنود 5–8 من الورقة (ظاهرة على الشاشة، لم تُحل في القسم).

- **5. حرارة الإنذار (Temperatura di Allarme).** اكتب برنامجًا بلغة C يطلب من المستخدم إدخال حرارة الغرفة. إذا كانت الحرارة أصغر من 18 **أو** أكبر من 26، يطبع البرنامج "Allarme temperatura"، وإلا يطبع "Temperatura OK".
- **6. لعبة في مدينة الملاهي (Giostra al Luna Park).** اكتب برنامجًا بلغة C يطلب من المستخدم الطول بالسنتيمتر والعمر. إذا كان الطول أكبر من أو يساوي 140 **و**العمر أكبر من أو يساوي 10، يطبع "Puoi salire" (يمكنك الصعود)، وإلا يطبع "Non puoi salire" (لا يمكنك الصعود).
- **7. عدد زوجي وموجب (Numero Pari e Positivo).** اكتب برنامجًا بلغة C يطلب من المستخدم إدخال عدد صحيح. إذا كان العدد أكبر من 0 **و**باقي قسمته على 2 يساوي 0 (`numero % 2 == 0`)، يطبع "Positivo e pari" (موجب وزوجي)، وإلا يطبع "Condizione non verificata" (الشرط غير محقق).
- **8. الدخول بكلمة مرور وPIN (Accesso con Password e PIN).** اكتب برنامجًا بلغة C يطلب من المستخدم إدخال رمز أمان وPIN (عددين صحيحين). فقط إذا كان الرمز يساوي 1234 **و**PIN يساوي 5678، يطبع "Accesso consentito" (الدخول مسموح)، وإلا يطبع "Accesso negato" (الدخول مرفوض).

12 ما يجب تذكره من هذا الدرس

- **"Oppure"** = **OR** (||)؛ **"e"** = **AND** (&&)؛ **"non"** = **NOT** (!)؛ **"diverso"** = =!.
- **السنة الكبيسة**: تقبل القسمة على 400، **أو** تقبل القسمة على 4 **ولا** تقبل القسمة على 100؛ تُفحص قابلية القسمة بأن يكون **الباقي** % مساويًا للصفر.
- **المثلث الصالح**: كل ضلع **أصغر من مجموع** الآخرين، ثلاث مقارنات يجمعها AND.
- **الصفر خطأ**، وكل عدد آخر **صحيح**؛ `allarme!` صحيح فقط إذا كانت قيمة `allarme` **صفرًا**.
- **الإزاحة** إلزامية: قوس الإغلاق تحت قوس الفتح، والمحتوى **جدولة واحدة إلى اليمين**؛ الكود بلا إزاحة **قيمته صفر**.
- **الأقواس العادية** دائمًا في الشروط المركبة؛ و**الأقواس المعقوفة** دائمًا، حتى مع تعليمة واحدة.
- **التعليقات**: /*… */ متعددة الأسطر، و// على سطر واحد؛ **المترجم** يتجاهلها؛ وهي في الشركات **إلزامية**.
- **Firmware**: برامج تشغل **اللوحات الإلكترونية**؛ و**المتحكم الدقيق** فيه **CPU** وذاكرتان (برنامج وبيانات).
- **Harvard**: ذاكرتان منفصلتان، أسرع (Arduino). **Von Neumann**: ذاكرة واحدة (الحاسوب الشخصي).

13 مسرد: كلمات إيطالية تُستخدم في القسم

- **anno bisestile**: سنة كبيسة
- **divisibile**: يقبل القسمة
- **lato / triangolo**: ضلع / مثلث
- **allarme attivo / disattivato**: إنذار مفعل / معطل

- **indentazione / tabulazione**: إزاحة / جدولة
- **commento**: تعليق
- **scheda**: ورقة أو لوحة (إلكترونية)
- **piedino**: رجل الشريحة
- **memoria programma / dati**: ذاكرة البرنامج / البيانات
- **relazione**: تقرير